

Step 2 — Add a Long-Term Store, and a Node That Reads From It — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 1's `state.py`, `graph.py`, and `main.py` already exist and work.

Basic Version

Approach 1 — the direct way, no duplicate-guard yet

```
# store.py
from langgraph.store.memory import InMemoryStore

_store = InMemoryStore()

def get_store():
    return _store

def namespace_for(user_id: str) -> tuple:
    return (user_id, "memories")
```

```
# graph.py (Basic -- recall_node always injects, no guard against repeating itself)
from functools import partial
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from state import ChatState
from store import namespace_for

model = ChatOpenAI(model="gpt-4o-mini")

def recall_node(state: ChatState, store) -> dict:
    namespace = namespace_for(state["user_id"])
    found = store.search(namespace, query="facts about this user")
    if not found:
```

```

        return {}
    facts_text = ", ".join(item.value.get("text", "") for item in found)
    return {"messages": [{"role": "system", "content": f"Known facts about this u

def chat_node(state: ChatState) -> dict:
    reply = model.invoke(state["messages"])
    return {"messages": [reply]}

def build_graph(checkpointer, store):
    builder = StateGraph(ChatState)
    builder.add_node("recall_node", partial(recall_node, store=store))
    builder.add_node("chat_node", chat_node)
    builder.add_edge(START, "recall_node")
    builder.add_edge("recall_node", "chat_node")
    builder.add_edge("chat_node", END)
    return builder.compile(checkpointer=checkpointer)

```

Expected output, after seeding `store.put(("sam", "memories"), "preferred_name", {"text": "Prefers to be called Sam"})` by hand and asking `"what's my name?"` on a brand-new thread: `"You told me your name is Sam."` This works, but calling the graph a second time on the *same* thread injects the known-facts system message again, since nothing checks whether it's already there.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a once-per-thread guard against duplicate injection

```
# graph.py (adds the guard)
def recall_node(state: ChatState, store) -> dict:
    namespace = namespace_for(state["user_id"])
    found = store.search(namespace, query="facts about this user")
    if not found:
        return {}

    already_injected = any(
        getattr(m, "type", None) == "system" and "Known facts about this user" in
        for m in state["messages"]
    )
    if already_injected:
        return {}

    facts_text = ", ".join(item.value.get("text", "") for item in found)
    return {"messages": [{"role": "system", "content": f"Known facts about this u
```

Expected output: calling the graph twice on the same thread now only injects the known-facts message once -- the first call adds it, the second call sees it's already present and adds nothing new, while the conversation itself still grows normally.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — main.py wired end to end, with a hand-seeded fact for testing

```
# main.py
import sqlite3
import uuid
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph
from store import get_store, namespace_for

USER_ID = "sam"

def main():
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    store = get_store()

    # seed a fact by hand, once, to prove recall works before Step 3 can write facts
    store.put(namespace_for(USER_ID), "preferred_name", {"text": "Prefers to be called Sam"})

    graph = build_graph(checkpointer, store)
    thread_id = str(uuid.uuid4()) # a genuinely new thread every run, unlike Step 1

    print(f"MemoryKeeper -- user_id={USER_ID}, thread_id={thread_id}. Type 'exit' to quit")
    while True:
        text = input("you: ")
        if text.strip().lower() == "exit":
            break
        config = {"configurable": {"thread_id": thread_id}}
        result = graph.invoke(
            {"messages": [{"role": "user", "content": text}], "user_id": USER_ID},
            config=config,
        )
        print("bot:", result["messages"][-1].content)

if __name__ == "__main__":
    main()
```

Expected output: run `python main.py`, immediately ask `what's my name? --` with **zero** prior messages on this brand-new `thread_id`, it still answers `"You prefer to be called Sam."` This is the actual proof Step 2 exists for: recall came from the Store, keyed by `user_id`, not from any conversation history, because there wasn't any yet.

Approach 2 — the automated test, extending Step 1's file

```
# test_memory_keeper.py (adds to Step 1's tests)
import sqlite3
import uuid
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph
from store import get_store, namespace_for

def ask_as_user(user_id: str, text: str, store) -> str:
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    graph = build_graph(checkpointer, store)
    thread_id = str(uuid.uuid4()) # a brand-new thread every single call, on pur
    config = {"configurable": {"thread_id": thread_id}}
    result = graph.invoke(
        {"messages": [{"role": "user", "content": text}], "user_id": user_id},
        config=config,
    )
    return result["messages"][-1].content

def test_new_thread_recalls_a_seeded_fact():
    store = get_store()
    store.put(namespace_for("priya"), "preferred_name", {"text": "Prefers to be c
    answer = ask_as_user("priya", "What's my name?", store)
    assert "priya" in answer.lower()

def test_unknown_user_gets_no_injected_facts_and_does_not_crash():
    store = get_store()
    answer = ask_as_user("brand-new-user-nobody-seeded", "What's my name?", store)
    assert "don't know" in answer.lower() or "not sure" in answer.lower() or "no"

if __name__ == "__main__":
    test_new_thread_recalls_a_seeded_fact()
    print("PASS: a brand-new thread recalls a fact saved for that user_id")
```

```
test_unknown_user_gets_no_injected_facts_and_does_not_crash()
print("PASS: a user with nothing saved gets a clean, honest answer, no crash")
```

Expected output:

```
PASS: a brand-new thread recalls a fact saved for that user_id
PASS: a user with nothing saved gets a clean, honest answer, no crash
```

Difference from Intermediate, and what this approach adds: Intermediate proves `recall_node` doesn't inject its message twice within one thread. Advanced proves the actual point of the step -- a `user_id` with a saved fact is recognized on a thread that has *never existed before*, and a `user_id` with nothing saved gets a clean, non-crashing answer instead of a `KeyError` or an empty, confusing injected message.

Which one should you actually write? Intermediate's duplicate-guard is worth keeping exactly as shown -- it's a small check that prevents a real, easy-to-miss bug once conversations run longer than a couple of turns. Write Advanced's `main.py` with a real generated `thread_id ( uuid.uuid4() )`, since a hardcoded one (Step 1's `THREAD_ID`) would accidentally let the checkpointer, not the Store, answer "what's my name?" and you'd never notice the Store wasn't actually doing the work. Keep both test functions -- the seeded-fact case and the unknown-user case -- since Step 3 will need `test_memory_keeper.py` to keep growing without you re-checking these two by hand every time.